Cloud Propensity Pipeline

Plain-language guide for everyone

This document explains—in everyday language—what the project does, how data moves through Google Cloud, and how running it on Vertex AI fits together. No programming background required.

1. What problem does this solve?

Imagine you have customer, insurance, and transaction information stored as spreadsheet-style files in the cloud. You want to **clean that data**, **build useful "features"** (patterns and summaries that help predict behavior), and **store those features in a database** (BigQuery) so analysts or machine-learning tools can use them later.

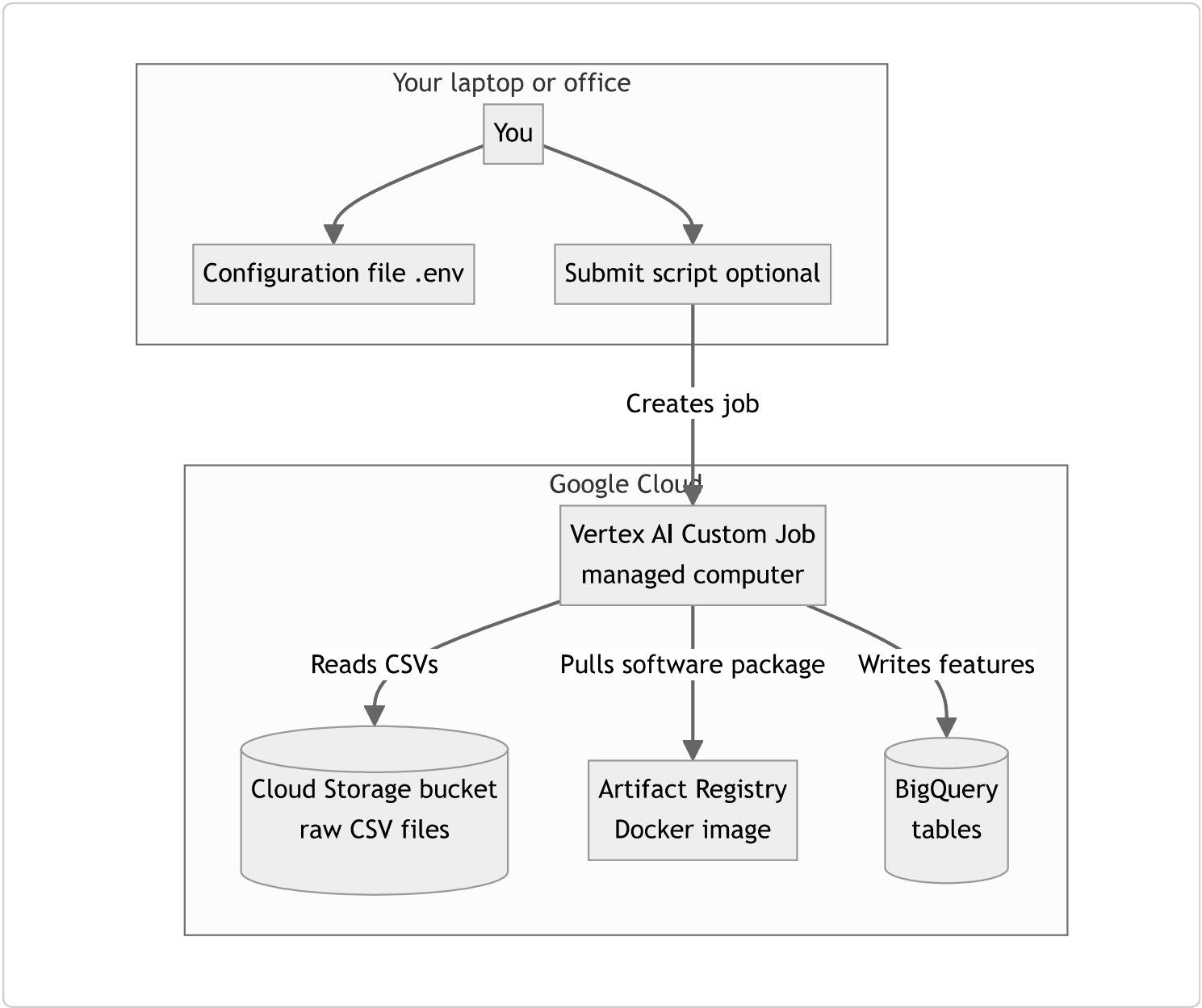
This project is an **automated assembly line** for that: it reads the files, processes them, and loads the results into BigQuery. Optionally, it can also pull short **history windows** (for example, 30 days before a key date) for specific customers.

2. The cast of characters (what each piece is)

Think of Google Cloud as a set of specialized services. Here is who does what:

Name	Simple analogy	Role in this project
Google Cloud Storage (GCS)	A giant, reliable filing cabinet in the cloud	Holds your **input CSV files** (customers, insurance, transactions). One bucket might be named like `propensitysyntheticdata`.
BigQuery	A very fast warehouse for huge tables	Stores the **output feature tables** (for example daily features and policy events) so you can query them with SQL.
Artifact Registry	A private "app library" for packaged software images	Stores the **Docker image**—a frozen copy of your code and its dependencies—that Vertex runs.
Vertex AI (Custom Job)	A button that says "run this packaged job on Google's computers"	Starts a machine, pulls your image, runs your pipeline script, then shuts down when done.
Cloud Build (optional)	A factory that builds the package for you	Can build the Docker image in the cloud and upload it to Artifact Registry—useful if you do not use Docker on your laptop.

3. Big picture: where does everything live?



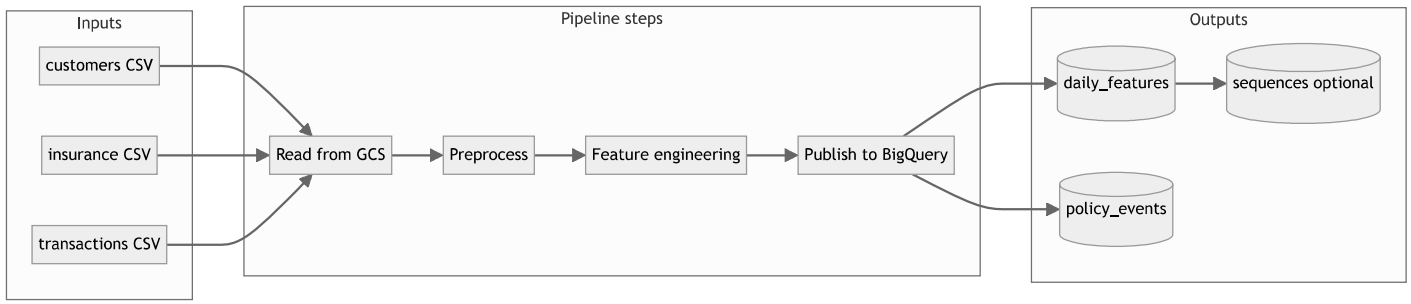
Important: Your **data** stays in the bucket you chose for CSVs. The **recipe** for running the pipeline is the Docker image in Artifact Registry. Vertex is simply the **kitchen** that follows the recipe.

4. The journey of your data (step by step)

When the pipeline runs—locally or on Vertex—the story is the same on the inside:

1. **Load** three CSV files from Cloud Storage (customers, insurance, transactions).
2. **Clean** and prepare the data (fix types, strip noise, align columns).
3. **Build features**—new tables that summarize behavior the way your project defines it.
4. **Load into BigQuery** so the results are queryable. Usually the job *replaces* the data in existing tables without deleting the whole table (safer for cloud permissions).

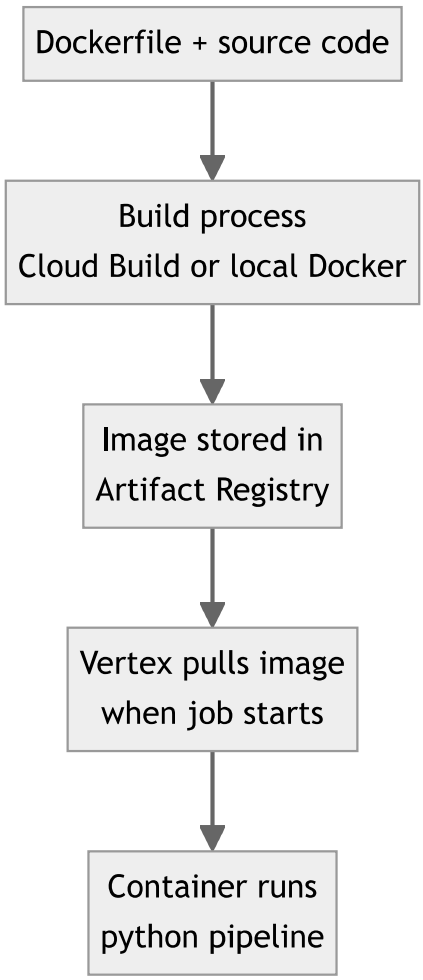
5. For the “full demo” script only: **sample a few customers**, then optionally build a **30-day sequence** table using SQL against the features already in BigQuery.



5. What is Docker, and why do we use it?

Docker image = a single, portable box that contains **your code**, the **Python libraries** it needs, and a **fixed command** to run (like “start the pipeline”). That way, Vertex always runs the **same environment**, no matter what is installed on your personal computer.

The **Dockerfile** in the project is the **blueprint** for that box. Cloud Build (or Docker on your machine) **builds** the box and **pushes** it to Artifact Registry.

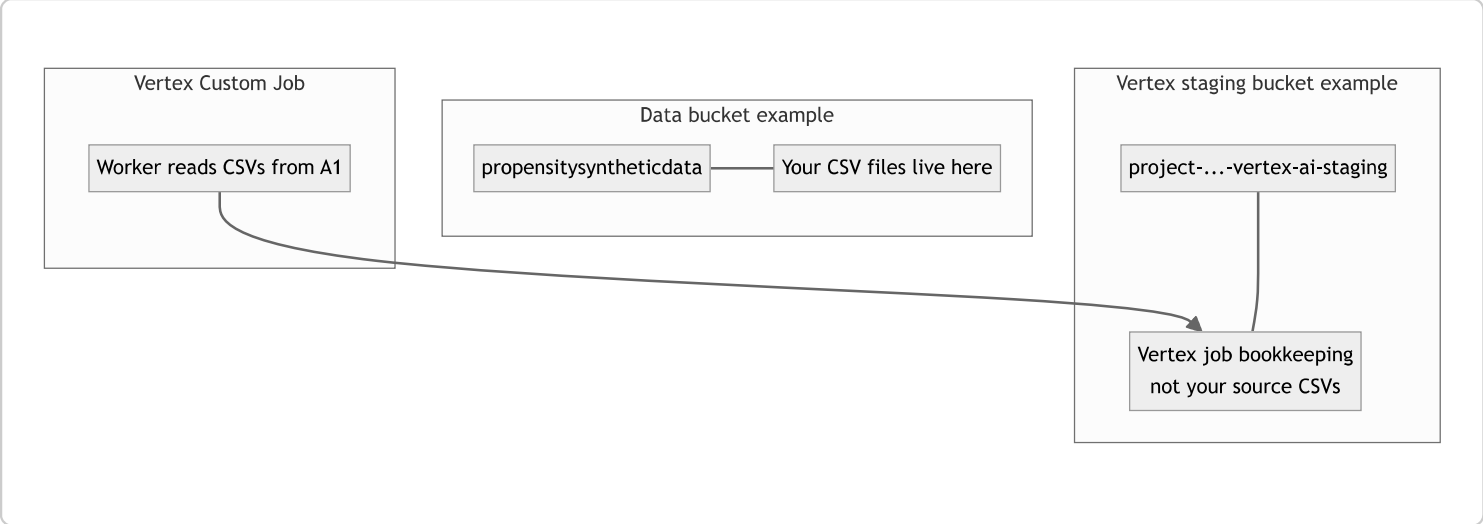


6. Artifact Registry in more detail

- **Where is the image?** In your Google Cloud project, in a **region** (for example `us-central1`), inside a **repository** (for example `cloud-propensity`).
- **What is the "address"?** A URI like `us-central1-docker.pkg.dev/PROJECT/REPO/IMAGE:tag`. That string is what Vertex uses to find the image.
- **Who can download it?** Google's systems use **service accounts** (robot users). Vertex's worker must have permission to **read** from Artifact Registry—usually straightforward when the image and job are in the same project.

7. Why two different buckets sometimes?

You might use one bucket for **your CSV data** (for example `propensitysyntheticdata`). Vertex AI also needs a **staging** place for its own job metadata and outputs tied to the **Custom Job API**. That staging bucket must sit in the **same regional location** as the Vertex region (for example `us-central1`). Many "data" buckets are multi-region **US**, which does *not* match a regional Vertex job—so a **second**, regional bucket is used only for Vertex staging. **Your CSVs are still read from the original data bucket.**



8. Running locally vs running on Vertex

	Local (your computer)	Vertex AI Custom Job
Who runs the code?	Your machine	Google-managed machine in the cloud
How do you start it?	Command like `python -m src.pipelines.build_features_pipeline`	Command that *submits* a job; Vertex runs the container entrypoint
Configuration	`.env` file on disk	Submit script passes key settings as **environment variables** into the container (project ID, bucket names, BigQuery names, etc.)
Best for	Quick tests, debugging	Repeatable production runs, large jobs, no laptop required

9. Why Custom Job and not “training pipeline”?

Vertex AI offers several patterns. **Custom Job** means: “run my containerized script on demand.” It is ideal for **data pipelines**, batch scoring, ETL, and feature generation—exactly what this repo does.

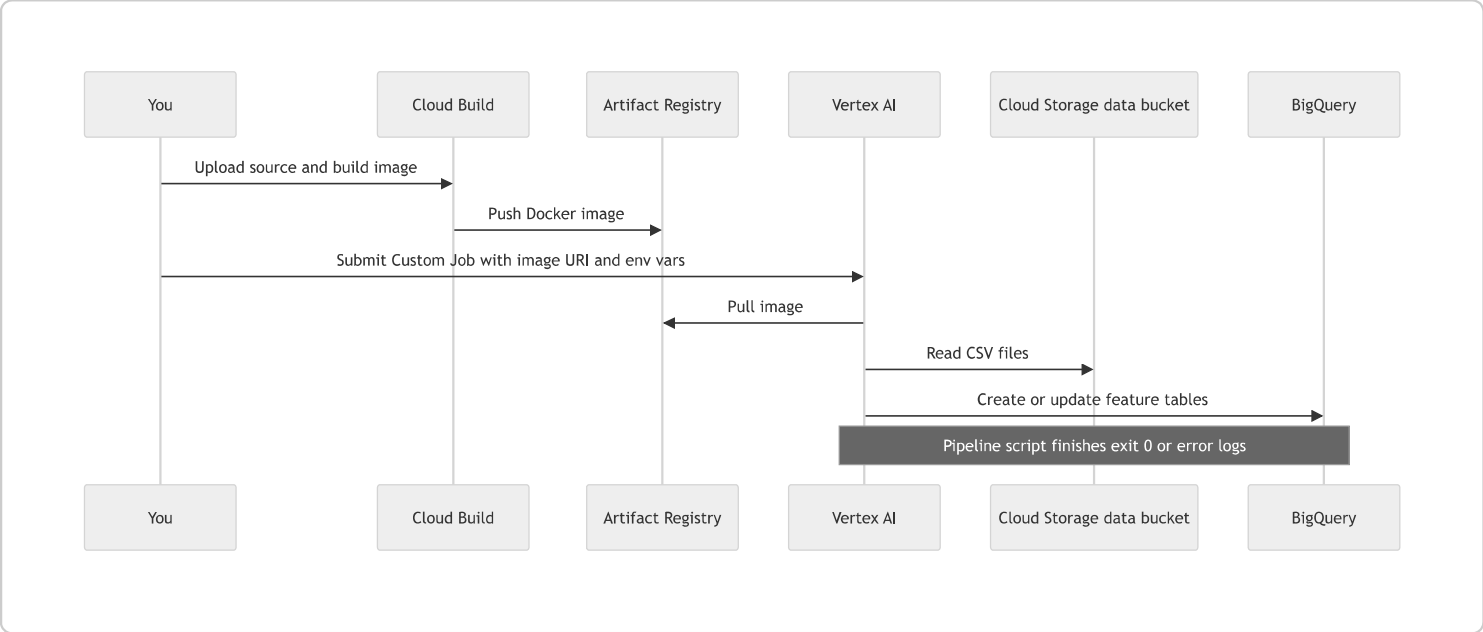
A **training pipeline** (and model-focused workflows) is optimized when the main output is a **trained model artifact** with experiment tracking. This project’s main output is **tables in BigQuery**, not a packaged model, so Custom Job is the natural fit. Later, if you split work into many steps and want a visual graph, retries, and lineage, you could adopt **Vertex AI Pipelines**—that is optional orchestration on top, not a requirement for correctness.

10. Configuration file (`.env`) in plain English

The `.env` file is a small list of settings the programs read at startup. Your `.env.example` shows the template; you copy it to `.env` and fill in real values. Typical entries:

Setting (concept)	What it means
Project ID	Which Google Cloud **project** owns BigQuery tables and billing.
Region	Where Vertex runs (for example `us-central1`).
GCS bucket / prefix	Where the **input CSVs** live.
BigQuery dataset and table names	Where **outputs** are written.
Vertex image URI	The Artifact Registry **address** of your Docker image.
Staging bucket (optional)	Regional bucket for Vertex job staging; defaults to a project-specific name if unset.
Recreate BigQuery tables (optional)	On Vertex, usually **off** so the service account does not need permission to delete tables; data can still be replaced using “truncate” style loads.

11. End-to-end workflow (Vertex path)



12. If something goes wrong (plain-language hints)

- **"Permission denied" on BigQuery** — The identity running the job needs roles to run queries and load data (and sometimes create datasets). A cloud admin assigns this in IAM.
- **"Permission denied" on Cloud Storage** — The job identity needs read access to the bucket that holds CSVs.
- **"Wrong project" in errors** — The container must receive the correct **project ID** via environment variables; the submit script is set up to inject them.
- **Staging bucket region mismatch** — Vertex needs a staging bucket in the **same region** as the job, not a multi-region "US" bucket.
- **Cannot delete BigQuery tables** — Cloud runs often disable "drop and recreate"; loads still refresh data using safer modes.

13. Glossary

Bucket
: A container in Cloud Storage that holds files and folders (prefixes).

Dataset (BigQuery)
: A folder-like grouping of tables inside a project.

Service account
: A special non-human user robots use to call Google APIs securely.

Custom Job
: Vertex runs your container once (or replicas) to completion—good for batch pipelines.

URI
: A standardized string that identifies a resource, like `gs://...` or `...docker.pkg.dev/...`.

Document purpose: Non-technical overview of the Cloud Propensity Pipeline and its Google Cloud workflow. Technical filenames and commands are shown only as reference; your team's exact values live in `.env` and the Google Cloud console.

Generated for the *cloud-propensity-pipeline* repository. To export as PDF, use Print → Save as PDF from Chrome or Edge after opening this HTML file.